

Document number: P3541R0
Date: 2024-12-16
Audience: SG21, SG23, EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Violation handlers vs noexcept

In this paper we point out that a number of correctness-checking features currently considered for addition to C++ that allow responding to runtime-detected violations via exceptions, should define the interaction with the `noexcept`-specifier and the `noexcept`-operator.

1. Background

There is a number of proposals currently on the flight aiming at detecting at run-time the incorrect usages of the language or libraries, and responding to them in a number of ways, including by throwing an exception. These proposals include:

1. [\[P2900R11\]](#) — Contracts for C++,
2. [\[P3471R0\]](#) — Standard library hardening,
3. [\[P3081R0\]](#) — Core safety profiles,
4. [\[P3100R1\]](#) — Undefined and erroneous behaviour is a contract violation.

In all these proposals we have the notion of a *violation handler*: a user-customisable function that is invoked in response to a bug detected at run-time at the point in program where the bug has been detected. One of the intended use cases for these handlers is for the programmer to be able to throw an exception that would on the one hand prevent the execution of the following expression, and on the other resume the program execution from another location.

2. The problem

The effect of these added runtime-checks is that expressions and constructs that used to be visibly non-throwing now can throw exceptions, possibly only under some compiler configurations. Consider the following function.

```
Tool::~~Tool() // noexcept by default
{
    for (int i = 0; i <= size(); ++i) {
        static_assert(noexcept(_vector[i]));
        _vector[i].retire(i);
    }
}
```

In C++23 this definition of a destructor compiles fine: the static assertion does not fire. But if a bounds profile is enforced, as per [\[P3081R0\]](#), and the programmer installs a throwing violation handler, the element-accessing expression can now throw an exception. So the question is, what should the `noexcept`-operator return in this case, and if the answer should depend on whether the bounds profile is enforced in the given context. Note also that even if the runtime check and the throw are performed outside of the expression, as suggested in [\[P3081R0\]](#), they are still performed inside the enclosing destructor, which is implicitly declared `noexcept`, so

an exception that is thrown doesn't achieve the goal of not terminating the program and still ends up calling `std::terminate()`.

In short, the idea of throwing an exception everywhere where we would previously have undefined behavior is clearly at odds with the semantics of `noexcept`, and also at odds with the notion of exception safety guarantees ([[ABRAHAMS](#)]).

While it is true that upon undefined behavior the language imposes nothing on the programs, and they are free to lie about `noexcept`, this is not helpful when the goal is to guarantee a predictable, non-surprising, non-UB, non-terminating behavior of an incorrect program.

3. The interaction with contracts

Contracts ([[P2900R11](#)]) are special in this regard, as the runtime-checks are never added implicitly but require an explicit usage of either `pre` or `post` on function declarations or `contract_assert` as a statement.

[[P2900R11](#)] clearly defines the interaction with `noexcept`. Preconditions and postconditions, even if physically checked in the caller, interact with `noexcept` as if they were evaluated inside the function body.

`contract_assert` is a statement rather than an expression, so that its non-throwing property cannot be tested in any way.

However, while this works for now, we know of two features that propose to be able to test the non-throwing property of arbitrary statements:

1. [[P3166R0](#)] — *Static Exception Specifications*, where you can annotate your function with `throw(auto)` which will deduce the exception specification from all statements in the function body,
2. [[P2806R2](#)] — *do expressions*, which turn a sequence of statements into an expression which you can inspect with the `noexcept`-operator.

If either of these features is added, the question will need to be answered: is `contract_assert` potentially throwing?

4. Conclusion

Given that the same idea — potentially throwing in a controlled way from every place that is undefined-behavior-like — surfaces in a number of proposals, the C++ community would benefit from a clear design direction in this respect.

The following are the possible directions that we can think of.

1. Any undefined behavior or contract assertion evaluating to `false` is potentially throwing and the throwing behavior can be controlled by programmers.
2. All these constructs are non-throwing, and we accept that "turn UB into a controlled throw" is not a thing.
3. The said features are either observably no-throwing or observably potentially throwing based on some configuration, maybe a compiler switch.
4. The said features are `noexcept(true)` but can throw nonetheless.

Any of the choices comes with a cost. #1 goes against the intuition of exception safety, where you need some operations to be never-throwing in order to provide a strong (transactional) exception safety guarantees. It also goes against the direction outlined in [P0709R4]. Note that a goal of turning UB into a throw requires exception safety to still be provided. #2 closes the door for the direction, where any bug could be turned into an exception and recovered from at some higher levels. #3 has a consequence of a correct program (even if it has no bugs and does no recovery) taking different paths of execution depending on the (conformant) compiler switches. #4, while allows throwing, makes the environment — the compiler and the libraries that depend on noexcept — unprepared for handling the exception in a reliable way, so the goal of reliably recovering from a detected bug. Options #1 and #2, while they close doors for certain use cases, have a clear, simple, teachable model to offer.

5. References

- [ABRAHAMS] — David Abrahams, "Exception-Safety in Generic Components" (https://www.boost.org/community/exception_safety.html).
- [N2855] — Douglas Gregor, David Abrahams, "Rvalue References and Exception Safety" (<https://www.open-std.org/JTC1/SC22/WG21/docs/papers/2009/n2855.html>).
- [N2983] — David Abrahams, Rani Sharoni, Doug Gregor, "Allowing Move Constructors to Throw" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2983.html>).
- [N3248] — Alisdair Meredith, John Lakos, "noexcept Prevents Library Validation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3248.pdf>).
- [P0709R4] — Herb Sutter, "Zero-overhead deterministic exceptions: Throwing values" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf>).
- [P2698R0] — Bjarne Stroustrup, "Unconditional termination is a serious problem" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2698r0.pdf>).
- [P2780R0] — Ville Voutilainen, "Caller-side precondition checking, and Eval_and_throw" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2780r0.html>).
- [P2806R2] — Bruno Cardoso Lopes, Zach Laine, Michael Park, Barry Revzin, "do expressions" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2806r2.html>).
- [P2858R0] — Andrzej Krzemiński, "Noexcept vs contract violations", (<https://isocpp.org/files/papers/P2858R0.html>).
- [P2900R11] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R11.pdf>).
- [P3081R0] — Herb Sutter, "Core safety Profiles: Specification, adoptability, and impact" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3081r0.pdf>).
- [P3100R1] — Timur Doumler, Gašper Ažman, Joshua Berne, "Undefined and erroneous behaviour is a contract violation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3100r1.pdf>).
- [P3166R0] — Lewis Baker, "Static Exception Specifications" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3166r0.html>).
- [P3471R0] — Konstantin Varlamov, Louis Dionne, "Standard library hardening" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3471r0.html>).